Software Testing, Quality Assurance and Maintenance	Winter 2026

Lecture 21 — March 23, 2026

Patrick Lam *version 1*

Under the hood with Dafny

Some of the things that Dafny does are a form of syntactic sugar which increase its usability. Let's look at this method again and desugar it, using what we know about Hoare Logic.

```
method add_by_inc(n: nat, m: nat) returns (r: nat)
  requires true
  ensures r = m + n
{
  r := n;
  var i := 0;
  while i < m
    invariant i ≤ m
    invariant r = n + i
  {
    i := i + 1;
    r := r + 1;
  }
  return r;
}
```

Dafny converts this into a bunch of verification conditions which it passes along to z3. So, first, let's convert the requires and ensures clauses into assumes/asserts. Also, to make our life easier, we'll combine the two invariants.

```
method add_by_inc(n: nat, m: nat) returns (r: nat)
{
  assume {:axiom} true;
  r := n;
  var i := 0;
  while i < m
    invariant i ≤ m ∧ r = n + i
  {
    i := i + 1;
    r := r + 1;
  }
  assert r = m + n;
  return r;
}
```

Remember that we had to show that the invariant held upon entry, that it was preserved by the loop; and we exit the loop with the invariant and not the loop condition. Let's do that manually here. First, that the invariant holds upon entry.

```
method add_by_inc(n: nat, m: nat) returns (r: nat)
{
  assume {:axiom} true;
  r := n;
  var i := 0;
  assert i ≤ m ∧ r = n + i;
  while i < m
    invariant i ≤ m ∧ r = n + i
```

```
  {
    i := i + 1;
    r := r + 1;
  }
  assert r = m + n;
  return r;
}
```

Next, we want to show that the invariant holds at the end of the loop body, if we know nothing but the invariant upon entry to the loop (at least about i and r). We stop the verification after the end of the loop body by putting assume false; no paths escape that assumption, so the verifier just does nothing further there.

```
method add_by_inc(n: nat, m: nat) returns (r: nat)
{
  assume {:axiom} true;
  r := n;
  var i := 0;
  assert i ≤ m ∧ r = n + i;
  // havoc i, r;
  i, r := *, *;
  assume {:axiom} i ≤ m ∧ r = n + i
  if i < m
  {
    i := i + 1;
    r := r + 1;
    assert i ≤ m ∧ r = n + i
    assume {:axiom} false;
  }
  assert r = m + n;
  return r;
}
```

Let's think about what we've done here. In fact, we've made Dafny check the three properties of inductive loop invariants.

1. Clearly, the first assert checks that it holds upon entry to the loop.
2. The havoc and assume erase knowledge of everything (havoc) but the loop invariant (assume) before the body of the loop executes. The loop body executes if the loop condition is true (and Dafny tracks the fact that the loop condition is true), and keeps the loop invariant. Then, Dafny checks that the loop invariant is preserved by the loop body. That's all that it checks for the loop body; it must not propagate facts further.
3. Finally, the do-nothing else branch of the if takes the invariant once again, and conjoins it with the negation of the loop condition, and continues from there; Dafny goes on and checks what happens after the loop exits—in particular, whether the postcondition exiting the loop is strong enough.

There was also an inferred decreases clause. We can use $\mathsf{m} - \mathsf{i}$ as the thing that decreases. It's easy to desugar that. We're going to use ghost variables; ghost means that a variable is specification-only—it doesn't need to be included in the executable. Nothing bad happens if you make them normal variables, but it's not as tidy.

We check that $\mathsf{m} - \mathsf{i}$ is smaller at the end of the loop than it was at the beginning of the loop, and also that it is nonnegative (the integers have infinite decreasing chains while the naturals don't; no infinite decreasing chain and strict decreasing together imply termination).

```
// as before..
{
  ghost var rank0 := m - i;
  i := i + 1;
  r := r + 1;
  ghost var rank1 := m - i;
  assert rank1 < rank0;
  assert rank1 ≥ 0;
  assert i ≤ m ∧ r = n + i
  assume {:axiom} false;
}
```

You can use this desugaring to help prove things. When Dafny is complaining about something, you can explicitly put in the assertions that it's trying to show, look at the code, and understand why the assertion fails. You can also put other assertions to figure out what else Dafny knows. And you can put in assumes to see which case Dafny can't show (we'll talk more about that).

In the dafny repository, you can find the above example as well as one with nested loops (file vcc.sol.dfy).

Fast exponentiation

This example is in fast_exp.prob.dfy and solved in fast_exp.sol.dfy in the Dafny repository. We will work through how we specify and verify the fast exponentiation algorithm, which uses repeated squaring to compute exponents; it uses the fact that

$$e^{2n} = (e^n)^2$$

i.e. to compute e^{2n} you can compute e^n and then square the result.

First, let's define a function exp. This part is straightforward; nothing to see here. The comments in the provided Dafny file provide the definition of exp that we're working from. This is a recursive definition.

```
function exp(x: real, n:nat) : real
{
  if n = 0 then 1.0
  else x * exp(x, n-1)
}
```

Next, we'd like to provide a specification for the provided implementation. Let's first look at the implementation and convince ourselves that it works.

```
method exp_by_sqr (x0: real, n0: nat) returns (r: real)
{
  if (n0 = 0) { return 1.0; }
  if (x0 = 0.0) { return 0.0; }
  var x, n, y := x0, n0, 1.0;
  while (n > 1)
  {
    if (n % 2 = 0)
    {
      x := x * x;
      n := n / 2;
    }
    else
    {
      y := x * y;
```

```
      x := x * x;
      n := (n-1)/2;
    }
  }

  return x * y;
}
```

We put aside the base cases, which are clear. In the even case, we see that we are repeatedly squaring x and halving n, which is exactly in line with the identity we saw above. In the odd case, we now have a variable y, which stores extra xs. Here, let's look.

$$x^{2n+1} = x \times x^{2n}$$

So we're assigning $x \times y$ to y, i.e. storing an extra factor of x here. And, in the end, we return x times the leftover ys that we've stored.

Dafny now has test cases, so let's also run a few test cases to convince ourselves that this implementation works. It can be useful to do that before we try to prove something that turns out to be not true. Instead of assert, which is statically verified, we use expect to write what we expect (shout-out to Kieran for finding this; the documentation on Dafny tests is hard to find.)

```
method {:test} TestExpBaseCases()
{
  var res1 := exp_by_sqr(17.0, 0);
  expect res1 = 1.0;
  var res2 := exp_by_sqr(2.2, 1);
  expect res2 = 2.2;
  var res3 := exp_by_sqr(0.0, 11);
  expect res3 = 0.0;
}

method {:test} TestExpEven()
{
  var res1 := exp_by_sqr(1.0, 4);
  expect res1 = 1.0;
  var res2 := exp_by_sqr(2.0, 4);
  expect res2 = 16.0;
  var res3 := exp_by_sqr(3.0, 2);
  expect res3 = 9.0;
}
```

```
method {:test} TestExpOdd()
{
  var res1 := exp_by_sqr(1.0, 5);
  expect res1 = 1.0;
  var res2 := exp_by_sqr(2.0, 5);
  expect res2 = 32.0;
  var res3 := exp_by_sqr(3.0, 3);
  expect res3 = 27.0;
}
```

We can run these test cases. Here I run them on a version of the program that I've verified, but you can also run tests on not-yet-verified code if you annotate relevant methods with {:verify false} and call dafny test --allow-warnings --no-verify.

```
$ dafny test fast_exp.dfy

Dafny program verifier finished with 9 verified, 0 errors
TestExpBaseCases: PASSED
TestExpEven: PASSED
TestExpOdd: PASSED
```

What's the specification? Well, we are returning r and it had better be equal to exp(x0, n0), so here we are:

```
  ensures r = exp(x0, n0)
```

Onwards to verification. As always, we at least need loop invariants (if not more). What could they be? Once again, we look at the postcondition, which essentially says x * y == exp(x0, n0). Then, Dafny gives two errors: (1) this invariant is not strong enough to prove the postcondition; and (2) the invariant is also not maintained by the loop. So now we have to debug verification issues.

It seems like it's a better bet to make sure the invariant is strong enough—might as well do that before we try to make it go through. Even if we expect it to be true, let's explicitly write out what we have upon exit to the loop, i.e. the negation of the loop condition plus the invariant. We can also explicitly write the postcondition as an assertion.

```
assert n ≤ 1 ∧ exp(x0, n0) = exp(x, n) * y;
assert x * y = exp(x0, n0)
```

Dafny can of course prove the first assertion but not the second one.

If we look at that first assertion, and we remember that Dafny starts its nat type at 0, then we might have $n = 0$. Is that indeed the case that is causing trouble? We can add an assume n == 1 to see. If we do that, then we see that Dafny is fine with the postcondition. Or, we can add assert n == 1 and see that Dafny can't prove that. We don't ever actually want to have $n = 0$ here, and we can realize that the loop invariant did not constrain n. This reminds us to add the usual kind of invariant on the loop variable:

```
invariant 1 ≤ n ≤ n0
```

and we can see that Dafny then only complains about the invariant being maintained by the loop body, but not about the postcondition.

Recall that what Dafny is effectively doing is putting an assume at the beginning of the loop body and an assert at the end. We can be explicit and put these, and indeed, Dafny complains at the end.

In the loop body, there is the n even case and the n odd case. We can focus on just one case by putting assume false on the other case, so that Dafny doesn't go on and try to verify that the invariant is maintained on that case. Let's handle the even case and put the assume on the odd case. (If we put it on both cases, then Dafny thinks it can verify the thing.) We can move our assert to the then case as well.

```
assume exp(x0, n0) = exp(x, n) * y;
if (n % 2 = 0)
{
  x := x * x;
  n := n / 2;
  assert exp(x0, n0) = exp(x, n) * y;
}
else
{
  assume false;
  y := x * y;
  x := x * x;
  n := (n−1)/2;
}
```

What about at the beginning of the then-clause? Of course we can put the loop invariant there. That will work, but it doesn't get us anywhere. But, do we know that exp(x0, n0) == exp(x*x, n/2) * y? We should! (This is what the loop body does). If we assert that, then Dafny says no. What if we move the assume into the body of the then-clause? Dafny still says no.

We can simplify this even more and just assert

```
assert exp(x, n) = exp(x*x, n/2);
```

and see that Dafny doesn't know about the key property of the exp function that we started this verification with.

We can go and remove extra asserts and assumes; we continue the verification of exp_by_sqr with just one assume:

```
if n % 2 = 0 {
  assume exp(x, n) = exp(x*x, n/2)
  //...
```

We can do the same process on the else branch to get the assume:

```
} else {
  assume exp(x, n) = exp(x*x, (n-1)/2) * x;
  //...
```

Using lemmas. Our proof so far has some assumes, which mean that we haven't quite proven correctness yet. We can look at the assume and say, well, of course it's true, but we don't really know. I'm going to deus ex machina some lemmas which we are going to use. Lemmas are functions that tell Dafny some additional facts about their parameters.

```
lemma exp_even (x:real, n:nat)
  requires n % 2 = 0
  ensures exp (x, n) = exp (x*x, n/2)
{
  // needs some code here rather than assume false
  assume false;
}

lemma exp_odd(x:real, n:nat)
  requires n % 2 = 1
  ensures exp (x, n) = exp (x*x, (n-1)/2)*x
{
  // needs some code here rather than assume false
  assume false;
}
```

Let's look at exp_even. If we call it from some other Dafny code, then, just like with a function, the precondition has to hold (i.e. n has to be even). Upon return from this lemma, we know the postcondition—the fact that exp (x, n) == exp (x*x, n/2). Which is just the property that we started with.

Lemmas are ghost code and don't get compiled into the executable. They exist only for their effects on the proof. Dafny takes them into account when it's verifying code. You also have to prove that the postcondition is actually true, but we will first use the lemma and then prove it. Here's what that looks like.

```
  if (n % 2 = 0)
  {
    exp_even(x,n);
    x := x * x;
    n := n / 2;
  }
  else
  {
```

```
    exp_odd(x,n);
    y := x * y;
    x := x * x;
    n := (n-1)/2;
  }
```

The precondition of the lemmas holds because it is just the immediately-preceding if condition. Dafny can therefore know that the postcondition is true. It happens that these postconditions are exactly the same as the assumes that we had previously inserted, so Dafny can verify the whole exp_by_sqr method (modulo the still-missing proofs of the lemmas).

Proving lemmas. Before we prove the lemmas for exp_by_sqr, we can learn how to prove some simpler lemmas. The file lemmas.prob.inclass.dfy contains some example lemmas which we'll work through.

We usually need lemmas when we have some function which Dafny doesn't know the properties of. Dafny is pretty good with properties of its + operator, but let's provide a add function which does the same thing. Dafny doesn't know that function.

```
function add (x: nat, y: nat): nat
{
  if (y = 0) then x
  else add (x, y-1) + 1
}

/* Lemma: x + 0 = 0 */
lemma add_zero_lemma (x: nat)
  ensures add (x, 0) = x
{
  // REPLACE BY PROOF
  assume(false);
}
```

It turns out that Dafny is actually smart enough to prove add_zero_lemma on its own, but let's provide our own proof of it, as a pedagogical exercise.

Dafny supports calc blocks, where we put a sequence of lines that are each equal to the next one. Using these blocks, we can supply calculational proofs.

```
lemma add_zero_lemma (x: nat)
  ensures add (x, 0) = x
{
  calc {
    add(x,0);
    x;
  }
}
```

This calc block shows that add(x,0) == x, which we knew already. If we make it wrong on purpose (e.g. add(x,1) == x), then Dafny complains.

This next lemma is actually harder to prove—it doesn't follow immediately from the definition of add. Dafny is still smart enough to prove it automatically, but we'll do it manually again.

```
lemma zero_add_lemma (x: nat)
  ensures add (0, x) = x
{
  assume(false);
}
```

We can split the proof into cases, e.g.

```
lemma zero_add_lemma (x: nat)
  ensures add (0, x) = x
{
  if x = 0 {
  } else if x = 1 {
  } else {
    assume false;
  }
}
```

Dafny verifies this, so we know that it can handle the 0 and 1 cases automatically. (These are add(0,0) == 0 and add(0, 1) == 1 respectively). Let's write a calc block for the other cases.

```
  calc {
    add(0,x);
    add(0, x-1)+1;
  }
```

What we really want to do now is to invoke induction, using 0 and 1 as the base cases. We can write the inductive hypothesis as an assume for calc:

```
  calc {
    add(0,x);
    add(0, x-1)+1;
    {zero_add_lemma(x-1);}
    (x-1)+1;
    x;
  }
```

Dafny checks that the preconditions to zero_add_lemma are met. Since there are none, those hold. Also, because it is a recursive call inside the braces, Dafny checks that the recursion is well-founded: the parameter x-1 is smaller than x, and there are base cases. We can then use the postcondition to replace add(0, x-1) with (x-1) in the calc block, and the rest of the calculation is easy.

There are a few more lemmas that Dafny can prove automatically. It's good to try to prove them manually as well, and the solutions are in lemmas.sol.dfy. I will put this one here, though.

```
lemma one_plus_add (x: nat, y: nat)
  ensures add(x, y) + 1 = add (x+1, y)
{ }
```

Here is one that Dafny cannot prove automatically.

```
/** Lemma: (x + y) = (y + x)
*/
lemma add_comm_lemma (x: nat, y: nat)
  ensures add(x, y) = add(y, x)
{
}
```

We get the usual error that there is a postcondition that could not be proved.

We can try some base cases:

```
/** Lemma: (x + y) = (y + x) */
lemma add_comm_lemma (x: nat, y: nat)
  ensures add(x, y) = add(y, x)
{
  if x = 0 { }
  else if y = 0 { }
  else {
```

```
  }
}
```

and see that those cases verify, but not the general case. Time to put a calc block.

```
  calc {
    add(x, y);
    add(x, y-1) + 1; // from the definition of add
```

This is straightforward so far. Then, we want to use induction again. The obvious thing to use induction on is what we have on the previous line, arguments (x, y-1). And we can then deploy the postcondition of the present lemma. Note that if you get the inductive arguments wrong, Dafny will indeed complain.

```
    {add_comm_lemma(x,y-1);}
    add(y-1,x) + 1;
```

Dafny can actually do it pretty much automatically from here, but let's explicitly also bring the +1 back into the add() parameters using induction again. The remaining steps after that are straightforward. We'll be pedantic about -1+1, though Dafny doesn't need that either.

```
    {one_plus_add(y-1,x);}
    add(y-1 + 1, x);
    add(y,x);
  }
```

Proving fast_exp lemmas. You don't actually need to use calc to prove these—you can iterate on n and write some invariants—but the solutions in fast_exp.sol.dfy do use calc and mutual recursion, and I guess that's nicer? You decide.

```
lemma exp_even (x:real, n:nat)
  requires n % 2 = 0
  ensures exp (x, n) = exp (x*x, n/2)
{
  if (n = 0) { }
  else
  {
    calc
    {
      exp (x, n);
      x * exp (x, n-1);
      { exp_odd (x, n-1); }
      x * x * exp (x*x, (n-2)/2);
      exp (x*x, n/2);
    }
  }
}

lemma exp_odd(x:real, n:nat)
  requires n % 2 = 1
  ensures exp (x, n) = exp (x*x, (n-1)/2)*x
{
  if (n = 0) { }
  else
  {
    calc
    {
      exp (x, n);
      x * exp (x, n-1);
      { exp_even (x, n-1); }
      x * exp (x*x, (n-1)/2);
```

```
    }
  }
}
```

Here are my iterative proofs. exp_odd just calls exp_even since, if n is odd, then n-1 is even; and I didn't use exp_odd in my exp_even proof.

```
lemma exp_even (x: real, n: nat)
  requires n % 2 = 0
  ensures exp (x, n) = exp (x*x, n/2)
{
  var i := 0;
  var r, rr := 1.0, 1.0;
  while i < n/2
    invariant 0 ≤ i ≤ n/2
    invariant r * r = rr
    invariant exp(x, i) = r
    invariant exp(x*x, i) = rr
    invariant exp(x, i*2) = rr
  {
    r := r * x;
    rr := rr * x * x;
    i := i + 1;
  }
}

lemma exp_odd(x: real, n: nat)
  requires n % 2 = 1
  ensures (exp (x, n) = exp (x*x, (n-1)/2)*x)
{
  exp_even(x, n-1);
}
```

Bubble Sort

We are now going to verify a bubble sort implementation, which will be helpful for doing the assignment. I am assuming that everyone here has seen bubble sort at some point in the past. It's been longer for me than it's been for you. (Though I wouldn't expect bubble sort to be in CS 341). And, spoiler, there is going to be a twist in the verification.

You can find the bubble sort implementation at bubble_sort.prob.dfy in the Dafny repo.

Auxiliary predicates: sorted and pivot. First, we are going to define a predicate sorted. We've done similar things before. This time we specify a from and to range to check for sortedness. The predicate returns true iff its array parameter a is sorted between the range [from, to).

```
predicate sorted(a: array<int>, from: int, to: int)
  requires 0 ≤ from ∧ to ≤ a.Length
  reads a
{
  ∀ j, k • from ≤ j < k < to ⟹ a[j] ≤ a[k]
}
```

Next, I'm going to introduce a pivot predicate. I'm going to do this before a reminder of how bubble sort works. You may remember pivots being useful, in general, for sorting algorithms.

pivot takes three arguments: an array, a to argument, and a potential pvt. This predicate checks the sub-array [0, to) of its array for whether pvt is a pivot—that is, all elements with index less than pvt are less than (or equal to) all elements of the sub-array with index greater than pvt.

We can encode the predicate as follows:

```
predicate pivot(a: array<int>, to: int, pvt: int)
  requires 0 ≤ pvt ≤ to ≤ a.Length
  reads a
{
  ∀ u, v • 0 ≤ u < pvt < v < to ⟹ a[u] ≤ a[v]
}
```

So, we let u take all values in $[0, \mathsf{pvt})$; and v is in $[\mathsf{pvt}+1, \mathsf{to})$. The predicate is true iff all a[u] are less than or equal to all a[v].

Dafny doesn't complain about this definition, but how do we know that the words correspond to the formalism?

We don't.

We can look at the formalism closely. It seems to be OK, but that is not a guarantee.

Another thing we can do is to run some test cases. I've included both assert and expect checks. In this case, in the Dafny model, probably assert is better: it is possible to verify these statically. For a method, all we check is the postcondition. But this is a function and so we are running the body when we use assert.

```
method {:test} TestPivot()
{
  var singleton := new int[] [4];
  assert pivot(singleton, singleton.Length, 0) = true;
  var three := new int[] [2,3,4];
  expect pivot(three, three.Length, 1) = true;
  var threeB := new int[] [4,3,2];
  expect pivot(threeB, threeB.Length, 1) = false;
  var a := new int[] [3, 5, 9, 2, 11, 13, 15, 12];
  expect pivot(a, a.Length, 0) = true;
  // a[3] < a[1], so false:
  expect pivot(a, a.Length, 2) = false;
  expect pivot(a, a.Length, 3) = true;
}
```

Dafny statically verifies the assert and, when you run the test case, it passes.

Implementing bubble sort. Here is an implementation of bubble sort. Note that it modifies array a in place.

```
method bubbleSort (a: array<int>)
{
  var i:nat := 1;

  while (i < a.Length)
  {
    var j:nat := i;
    while (j > 0)
    {
      if (a[j-1] > a[j]) {
        // Swap a[j-1] and a[j]
        var temp := a[j-1];
        a[j] := temp;
```

```
        a[j-1] := a[j];
      }
      j := j - 1;
    }
    i := i+1;
  }
}
```

Let's now review a bit how this works. We know that bubble sort is $O(n^2)$. The outer loop ensures that the subarray from $[0, i)$ is sorted at each iteration. The inner loop makes this happen by bubbling down items inside $[0, i)$ that are out-of-order, starting at index i and pushing them as far as needed, possibly to index 0.

For instance:

```
9, 3, 5, 2
3, 9, 5, 2
3, 5, 9, 2
3, 5, 2, 9
3, 2, 5, 9
2, 3, 5, 9
```

In the lecture notes I'm not showing the loop iterations explicitly, but it's a worthwhile exercise to write the array contents at each stage, and in particular, to note which sub-arrays are sorted.

Let's write a contract. This is perhaps an obvious contract for this method.

```
requires a.Length > 0
ensures sorted(a, 0, a.Length)
modifies a
```

And we know that we have to write invariants for the while loop. We've done this several times already, so we know that this invariant is likely to work:

```
invariant i ≤ a.Length
invariant sorted(a, 0, i)
```

As we might expect, because we haven't provided any invariants at all for the inner loop, all Dafny knows coming out of the inner loop is that $j = 0$, which of course does not imply that any sub-array is sorted.

What I did next was look at the expanded loop iterations to see which sub-arrays were sorted. One pattern that is possible to see is that, of course, the sub-array $[0, i)$ always remains sorted in the inner loop; and also, sub-array $[j, i + 1)$ also remains sorted. So we can write this inner loop invariant:

```
invariant 0 ≤ j ≤ i ≤ a.Length
invariant sorted(a, 0, i)
invariant sorted(a, j, i+1)
```

and Dafny doesn't complain.

It might seem like we're done. That was easy?

Are you sure? Let's write a test case.

```
method {:test} TestSort()
```

```
{
  var a := new int[] [3, 5, 9, 2, 11, 13, 15, 12];
  bubbleSort(a);
  var b := new int[] [2, 3, 5, 9, 11, 12, 13, 15];
  expect a[..] = b[..];
}
```

Dafny's arrays, like many other languages, are object-like. It's like Java. Comparing == of two arrays is probably not what you want, and definitely not here. We want something like Java's equals(), where we check the contents of the arrays. To do that in Dafny, we can form sequences (a Dafny built-in type) from the arrays, and compare those.

If we run this test case, Dafny reports that it fails. (Since we are calling a method, this really has to be expect and not assert). Why is this? The test harness output is not so helpful, but we can add print a[..] to the test case, printing out the sequence:

```
Dafny program verifier finished with 10 verified, 0 errors
TestPivot: PASSED
TestSort: [3, 5, 9, 9, 11, 13, 15, 15]FAILED
  bubble_sort.prob.dfy(88,1): expectation violation
TestSort2: PASSED
[Program halted] bubble_sort.prob.dfy(12,0): Test failures occurred: see above.
```

Uh oh. That's not what we expected. Somehow there are two 9s and two 15s in the array.

Multisets to the rescue. We have seen that the output is wrong. Even though it verifies. The array is indeed sorted, but it's not at all equal to a permutation of the original array. Our ensures clause is too weak, and forgot to specify that part of the relationship between the old array and the new array. Indeed, a function could just return the empty array and still satisfy the specification.

Permutations are actually not so obvious in Dafny. But Dafny has primitive type multiset, which stores elements and their multiplicities. If the output array is sorted, and if converting both arrays to multisets and comparing them succeeds, then we can say that we have a sort implementation. Let's do that.

```
method bubbleSort2 (a: array<int>)
  requires a.Length > 0
  ensures sorted(a, 0, a.Length)
  ensures multiset(a[..]) = multiset(old(a[..]))
  modifies a
```

Here, old refers to the contents of a upon entry to the method. Dafny has to be able to reason about those while verifying, so that we can compare old and new.

Of course, bubbleSort2 doesn't verify. It shouldn't! We add the multiset constraint as an invariant to both loops as well. Now, Dafny says that the multiset invariant is not preserved by the inner loop. That's odd... the contents are changing in the inner loop? Oh! It turns out that someone put wrong code to swap array elements. There was a bug. Despite verification (of the wrong contract). Let's fix that.

```
  var temp := a[j-1];
  a[j-1] := a[j];
  a[j] := temp;
```

Now Dafny doesn't complain about the multiset invariant anymore. It does complain about how sortedness from 0 to i is not maintained by the loop.

I did mention the pivot predicate a while ago. And the first sorted call from 0 to i seems somewhat fishy. It would make more sense to have that call range from 0 to j, which is the loop variable here. And it would also make sense for a[j] to be the pivot of the sub-array between 0 and $i + 1$. So, this invariant actually works for the inner loop:

```
invariant 0 ≤ j ≤ i ≤ a.Length
invariant sorted(a, 0, j)
invariant pivot(a, i+1, j)
invariant sorted(a, j, i+1)
invariant multiset(a[..]) = multiset(old(a[..]))
```

That's all the insight I can give you about where the corrected invariants come from.

Dafny does verify this! It would be smart to still be a bit suspicious. But we can cross-check the verification with a test case.

```
method {:test} TestSort2()
{
  var a := new int[] [3, 5, 9, 2, 11, 13, 15, 12];
  bubbleSort2(a);
  var b := new int[] [2, 3, 5, 9, 11, 12, 13, 15];
  expect multiset(a[..]) = multiset(b[..]);
}
```

which passes. Between the test case (which is not super strong evidence) and the verification, we can be more sure that this implementation is correct, though again, there are still no guarantees.

We are still pretty confident in our bubble sort implementation. We did, however, use multisets. This is not a correctness issue, but it's good to know how to implement things ourselves.

Using sequences rather than multisets

We are instead going to use sequences, which we briefly mentioned in Lecture 16. As we (briefly) saw back there, Dafny can seamlessly convert arrays into sequences; we'll use sequences in our lemmas. A Dafny seq$\langle T \rangle$ is an immutable sequence of elements of type T; for example, seq$\langle$int$\rangle$ is a sequence of ints. Unlike arrays and lists, but more like a Java String, they are immutable. Recall that if 'a' is an array then 'a[..]' is a sequence with the contents of the array.

Let's start with some helper functions. Note that since a seq is a functional datatype, we can't modify it, but we can easily take a sub-sequence with a[1..].

```
/**
  Returns number of occurrences of a given value `val` inside a given sequence
 */
function count(a:seq<int>, val:int): nat
{
  if |a| = 0 then 0
  else
    if a[0] = val then 1 + count(a[1..], val)
    else count(a[1..], val)
}
```

Dafny knows that this function terminates—the length of count is decreasing in recursive calls.

We use this function to define a predicate which says whether a is a permutation of b—recall that this was the main problem in our incomplete specification of bubble sort.

```
/**
  A sequence `b` is a permutation of sequence `a` if every integer `v` has
  exactly the same number of occurrences in `a` and in `b`
 */
ghost predicate perm(a: seq<int> , b: seq<int>)
{
  ∀ v • count(a, v) = count(b, v)
}
```

Dafny complains that perm is not compilable unless we label it a ghost predicate, which means that we can use it in specifications but not in code. That's all we need anyway.

We know some things about permutation. Dafny can figure them out, but we help it by expressing them.

```
/** Permutation is transitive.
    Dafny can prove that automatically.
 */
lemma trans_perm(a: seq<int>, b: seq<int>, c: seq<int>)
  requires perm(a, b) ∧ perm(b, c)
  ensures perm(a, c)
{}

/** Permutation is commutative.
    Dafny can prove that automatically.
  */
lemma comm_perm(a: seq<int> , b: seq<int> )
  requires perm(a, b)
  ensures perm(b, a)
{}
```

Here's a lemma about swapping two positions in a sequence. Dafny cannot prove it automatically. I'll leave this as an exercise.

```
/**
   If sequences `a` and `b` differ only in two positions, and the values in
   these to positions are swapped, then sequences are permutations of one
   another.

   This is assumed without a proof. Completing the proof is left as an exercise.
 */
lemma swap_lemma(a: seq<int>, b: seq<int>, pos1: nat, pos2: nat)
  requires |a| = |b|
  requires 0 ≤ pos1 < pos2 < |a|
  requires ∀ k • 0 ≤ k < |a| ∧ k ≠ pos1 ∧ k ≠ pos2 ⟹ a[k] = b[k]
  requires a[pos1] = b[pos2]
  requires a[pos2] = b[pos1]
  ensures perm(a, b)
  { assume(false); }
```

We use the following method in our bubble sort. Here we specify it in isolation:

```
/** A helper method to swap two positions in an array */
method swap(a: array<int>, pos1: nat, pos2: nat)
  modifies a
  requires 0 ≤ pos1 < pos2 < a.Length
  ensures ∀ k • 0 ≤ k < a.Length ∧ k ≠ pos1 ∧ k ≠ pos2 ⟹ a[k] = old(a[k])
  ensures a[pos1] = old(a[pos2])
  ensures a[pos2] = old(a[pos1])
{
  var temp := a[pos2];
  a[pos2] := a[pos1];
```

```
  a[pos1] := temp;
}
```

Now here is an implementation of bubble sort using the helper method and the lemmas. We do a lot of the work for Dafny here—Dafny doesn't know these properties of permutations by default.

```
/**
   Verified version of bubble sort, in which the prove that the output is
   permutation of the input is done explicitly without using any special help
   from Dafny.
 */
method bubbleSort3 (a: array<int>)
  requires a.Length > 0
  ensures ∀ u, v • 0 ≤ u < v < a.Length ⟹ a[u] ≤ a[v]
  ensures perm(a[..], old(a[..]))
  modifies a
{
  var i:nat := 1;

  while (i < a.Length)
    invariant i ≤ a.Length
    invariant ∀ u, v • 0 ≤ u < v < i ⟹ a[u] ≤ a[v]
    invariant perm(a[..], old(a[..]))
  {
    var j:nat := i;
    while (j > 0)
      invariant 0 ≤ j ≤ i
      invariant ∀ u, v • 0 ≤ u < v < j ⟹ a[u] ≤ a[v]
      invariant ∀ u, v • 0 ≤ u < j < v ≤ i ⟹ a[u] ≤ a[v]
      invariant ∀ u, v • j ≤ u < v ≤ i ⟹ a[u] ≤ a[v]
      invariant perm(a[..], old(a[..]))
    {
      ghost var old_a := a[..];
      if (a[j-1] > a[j]) {
        // Swap a[j-1] and a[j]
        swap(a, j-1, j);
        swap_lemma(old_a, a[..], j-1, j);
        comm_perm(old_a, a[..]);
      }
      j := j - 1;

      trans_perm(a[..], old_a, old(a[..]));
      assert(perm(a[..], old(a[..])));
    }
    i := i+1;
  }
}
```

Here we add the postcondition we were missing before in our naïve initial try. The invariants are also similar to the previous non-refactored example, except that we use perm instead of the multisets that we had invoked before. The swap operation itself, though, also uses the swap_lemma and the commutativity property comm_perm. Finally, we also point out to Dafny that perm is transitive at the end of the outer loop.

It turns out, though, that Dafny actually only really needs the swap_lemma.

Bounding runtimes

Dafny can do that too: it can prove that this bubble sort implementation is $O(|a|^2)$. We add some instrumentation to our earlier implementation and assert on the instrumentation's values.

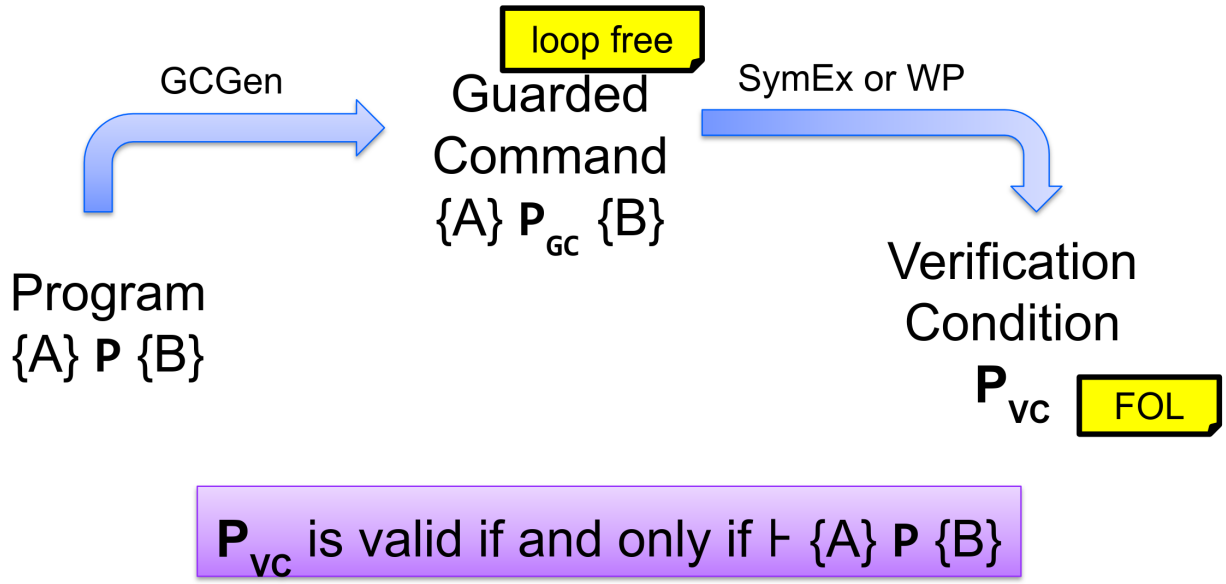
```
method bubbleSort4 (a: array<int>)
  requires a.Length > 0
  ensures ∀ u, v • 0 ≤ u < v < a.Length ⟹ a[u] ≤ a[v]
  modifies a
{
  // counter that counts number of executions of the main loop
  var cnt := 0;
  var i:nat := 1;

  while (i < a.Length)
    invariant i ≤ a.Length
    invariant ∀ u, v • 0 ≤ u < v < i ⟹ a[u] ≤ a[v]
    invariant cnt ≤ a.Length * i
  {
    var j:nat := i;
    while (j > 0)
      invariant 0 ≤ j ≤ i
      invariant ∀ u, v • 0 ≤ u < v < j ⟹ a[u] ≤ a[v]
      invariant ∀ u, v • 0 ≤ u < j < v ≤ i ⟹ a[u] ≤ a[v]
      invariant ∀ u, v • j ≤ u < v ≤ i ⟹ a[u] ≤ a[v]
      invariant cnt ≤ a.Length * i + (i-j)
    {
      if (a[j-1] > a[j]) {
        // Swap a[j-1] and a[j]
        var temp := a[j-1];
        a[j] := temp;
        a[j-1] := a[j];
      }
      j := j - 1;
      // increment count each iteration of the loop
      cnt := cnt + 1;
    }
    i := i+1;
  }
  // claim that cnt is bounded above by |a|^2
  assert(cnt ≤ a.Length * a.Length);
}
```

Verification Condition Generation

Moving on, and continuing on the implementing-things-ourselves theme, we are going to talk more about how Dafny works under the hood—earlier we had talked about loops, now we'll talk about normal statements. This is called *verification condition generation*. Here's a picture, which should not be a surprise.

Verification Condition Generation in a Nutshell



Step 1: Loop-Free Guarded Commands

For verification purposes, we first compile the program into loop-free guarded commands. Here's the grammar for these commands.

```
c ::= assume b
  |   assert b
  |   havoc x
  |   c1; c2
  |   c1 || c2
```

The guards are the asserts and assumes. You can't execute a program once it's been compiled to guarded commands, but proving correctness of the guarded commands is equivalent to proving correctness of the original.

Let's look at the rules to translate WHILE into guarded commands. This is similar to Hoare Logic.

- GC(skip): assume true
- GC(x := e): assume tmp = x; havoc x; assume (x = e[tmp/x]), where tmp is fresh.
- GC(c_1; c_2): GC(c_1); GC(c_2)
- GC(if b then c_1 else c_2): (assume b; GC(c_1) || assume $\neg b$; GC(c_2)
- GC(while b inv I do c): ...

The rule for assignment is like the one for Hoare Logic. We zero out x and then assume that it is e, taking care to not use the x that we're defining if it should occur in e, but rather old x.

For if, it looks like we have parallel composition, but in any actual execution, either b is concretely true or concretely false. So one of the branches has what amounts to assume false and verification would stop on that branch in that condition. The execution continues on the branch that does not have an assume false.

We've also seen the translation for loops:

- GC(while b inv I do c):

```
assert I;
havoc x_1; ...; havoc x_n;          // (variables modified in c)
assume I;
((assume b; GC(c); assert I; assume false) ||
   assume ¬b)
```

Example. Let's look at how this works on our multiplication example again.

```
{n >= 0}
p := 0;
x := 0;
while x < n inv p = x * m && x <= n do
  x := x + 1;
  p := p + m
{p = n * m}
```

The conversion is mechanical and we get this:

```
{ n >= 0 }
assume p0 = p; havoc p; assume p = 0;
assume x0 = x; havoc x; assume x = 0;
assert p = x * m && x <= n;
havoc x; havoc p; assume p = x * m && x <= n;
  ((assume x < n;
    assume x1 = x; havoc x; assume x = x1 + 1;
    assume p1 = p; havoc p; assume p = p1 + m;
    assert p = x * m && x <= n; assume false)
   || assume x >= n)
{ p = n * m }
```

For the assignment statements, we can see both the cases where there is no capture and where an assignment refers to the old value of, for instance, `x`—it introduces temporary variable `x1`. For the while loop, we assert the invariant before the loop, then clear everything but the invariant, assume it, and show it at the end of the loop body. When exiting the loop, we assume the negation of the loop body.

Step 2: Verification Condition Generation

Here are two ways we could proceed.

Idea 1: Exhaustive symbolic execution of GC program. Symbolic execution sometimes appears in this course, but I've removed it this year. Basically, instead of tracking concrete values of program variables, you track abstract (symbolic) values. It is, in principle, possible to use symbolic execution to verify the guarded commands: start at the beginning, keep on tracking the symbolic state, and see if we can prove the postcondition at the end. The program is correct if no assertion can fail.

The symbolic execution is implicitly constructing verification conditions and bringing them through to the end.

Because of the way this works, we start with the pre-condition, and use the program structure, but here we are bringing *everything* through to the end, even if that is much more than what is needed to prove the postcondition. The symbolic state, for instance, can get exponentially huge, and we'd rather not have that. Anyway, we won't talk about this more.

Idea 2: Propagate the post-condition backwards through the program. Here, the idea is to use the post-condition and proceed backwards, checking that the required pre-condition, plus the program behaviour, is strong enough to ensure the post-condition.

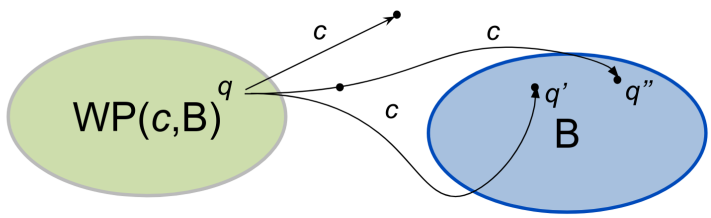
That is, for a Hoare triple $\{A\}P\{B\}$, we generate FOL formula $A \Rightarrow F(P, B)$, where backwards propagation $F(P, B)$ is formalized in terms of *weakest preconditions*.

This approach has always seemed less intuitive to me than the symbolic execution approach, but it does work better.

The weakest precondition $\mathrm{WP}(c, B)$ holds for any state q whose c-successor states all satisfy B; that is,

$$q \models \mathrm{WP}(c, B) \quad \text{iff} \quad \forall q' \in Q.\ q \xrightarrow{c} q' \Rightarrow q' \models B$$

Or, as a picture:



It really is backwards, but effective.

WP Examples. Let's again work through some examples. This is similar to the Hoare Logic examples we saw earlier. It's a mechanical process to start with, but then also we can simplify the formulas.

$\{\quad\}$	$x := z + w$	$\{x \geq y\}$
$\{\quad\}$	if $y > 0$ then $x := x + 1$ else $x := y + 4$	$\{x \geq 42\}$
$\{\quad\}$	if $y > 0$ then $x := z$ else $x := y$	$\{x \geq y + z\}$

For the first one, we push through the RHS to obtain weakest precondition

$$z + w \geq y$$

If you think about it for a bit—or, better yet, work through some examples—you can see that anything weaker is not going to give the needed postcondition, but that what we have here does.

We can mechanically push backwards $x \geq 42$ in the second case, getting a disjunction:

$$(y > 0 \wedge x + 1 \geq 42) \vee (y \leq 0 \wedge y + 4 \geq 42)$$

which simplifies to

$$(y > 0 \wedge x \geq 41) \vee (y \leq 0 \wedge y \geq 38)$$

and we can drop the second disjunct, which can never be true, to get weakest precondition:

$$y > 0 \wedge x \geq 41$$

Similarly, in the third case, the mechanical process yields:

$$(y > 0 \wedge z \geq y + z) \vee (y \leq 0 \wedge y \geq y + z)$$

which simplifies to

$$(y > 0 \wedge 0 \geq y) \vee (y \leq 0 \wedge 0 \geq z)$$

also known as

$$(y = 0) \vee (y \leq 0 \wedge z \leq 0)$$

This is all mechanical and Dafny does it for you.

So, now we can compute weakest pre-conditions. How does that help for program verification? We have a method, along with a post-condition, and compute the weakest pre-condition. If the method pre-condition is stronger than the weakest pre-condition, then any valid way that you call the method is going to ensure the stated post-condition. This verifies the method.

(Another way of saying this is that, if you put the pre-condition as an assume at the beginning of the method, then you have to compute the weakest pre-condition as true; otherwise, anything that is "left over" is a way to break the function contract.)

Computing Weakest Pre-conditions. Here are rules for mechanically computing the weakest preconditions.

- WP(assume b, B): $b \Rightarrow B$
- WP(assert b, B): $b \wedge B$
- WP(havoc x, B): $B[a/x]$ (a fresh in B; "replace x by a")
- WP(c_1 ; c_2, B): WP(c_1, WP(c_2, B))
- WP(c_1 || c_2, B): WP(c_1, B) $\wedge$ WP(c_2, B)

Let's start with c_1 || c_2. In that case, the weakest precondition has to be the conjunction of those for c_1 and c_2, because either of those (or both, in some sense) could execute, so we'd better account for them.

Conversely, for sequential composition, we calculate the WP for c_2 first, and then take its output, go backwards, and compute the weakest precondition for c_1. So you can get to the start of c_1 by going through c_2 and then c_1.

For a havoc statement, we wipe out everything we knew about x and put in some fresh a to replace it. B has to be true no matter what x is afterwards. The new a can pick up more constraints as the computation continues backwards through the method.

The weakest precondition for an assert is that the expression b being asserted must hold as part of the weakest precondition. Otherwise, the execution aborts. Note that this doesn't change the state.

Finally, for an assume statement, we use the property of implication where we make no guarantees about what happens if the antecedent is false. So, if the antecedent is true (the assumption holds), then we say the incoming thing B; otherwise, we basically quietly abort the execution and pretend nothing happened. (There is a subtle point about partial correctness here; the requirement is just that there *exists* an execution that has the desired effect, not that all executions do, so assume can drop executions and it's fine.)

Putting everything together. This is what Dafny does:

Given a Hoare triple $H \equiv \{A\}P\{B\}$,

- compute $c_H :=$ assume A; $\mathrm{GC}(P)$; assert B;
- compute $\mathrm{VC}_H := \mathrm{WP}(c_H, \mathsf{true})$; and,
- prove $\vdash \mathrm{VC}_H$ using a theorem prover.

One last example. But it's a bit of a doozy. Let's compute the weakest precondition of the multiplication program, which we'd already converted to the guarded command language.

```
WP( assume n ≥ 0;
    assume p0 = p; havoc p; assume p = 0;
    assume x0 = x; havoc x; assume x = 0;
    assert p = x * m ∧ x  ≤  n;
    havoc x; havoc p; assume p = x * m ∧ x  ≤  n;
        ((assume x < n;
          assume x1 = x; havoc x; assume x = x1 + 1;
          assume p1 = p; havoc p; assume p = p1 + m;
          assert p = x * m ∧ x  ≤  n; assume false)
        || assume x ≥ n) ;
    assert p = n * m, true)
```

We have incorporated the precondition and postcondition already, as an assume/assert pair, so we aim to finish with weakest precondition true.

We start at the end with the assert statement, which got tacked on to the end using sequential composition. WP for the assert is `p = n * m` and applying the rule for `;` gives us new B being the same expression:

```
WP( assume n ≥ 0;
    assume p0 = p; havoc p; assume p = 0;
    assume x0 = x; havoc x; assume x = 0;
    assert p = x * m ∧ x ≤ n;
    havoc x; havoc p; assume p = x * m ∧ x ≤ n;
        ((assume x < n;
          assume x1 = x; havoc x; assume x = x1 + 1;
          assume p1 = p; havoc p; assume p = p1 + m;
          assert p = x * m ∧ x ≤ n; assume false)
        || assume x ≥ n), p = n * m)
```

Continuing backwards, the next sequential composition is between

```
assume p = x * m ∧ x ≤ n
```

and what we got converting the while body, so we split this and compute the weakest preconditon of the loop body separately; it'll be the input to computing WP on everything before that.

```
WP( assume n ≥ 0;
    assume p0 = p; havoc p; assume p = 0;
    assume x0 = x; havoc x; assume x = 0;
    assert p = x * m ∧ x ≤ n;
    havoc x; havoc p; assume p = x * m ∧ x ≤ n,
      WP((assume x < n;
          assume x1 = x; havoc x; assume x = x1 + 1;
          assume p1 = p; havoc p; assume p = p1 + m;
          assert p = x * m ∧ x ≤ n; assume false)
        || assume x ≥ n, p = n * m))
```

We have a ||, so we change that into a conjunction.

```
WP( assume n ≥ 0;
    assume p0 = p; havoc p; assume p = 0;
    assume x0 = x; havoc x; assume x = 0;
    assert p = x * m ∧ x ≤ n;
    havoc x; havoc p; assume p = x * m ∧ x ≤ n,
      WP(assume x < n;
          assume x1 = x; havoc x; assume x = x1 + 1;
          assume p1 = p; havoc p; assume p = p1 + m;
          assert p = x * m ∧ x ≤ n; assume false, p = n * m)
    ∧ WP(assume x ≥ n, p = n * m))
```

Let's convert the last assume into an implication.

```
WP( assume n ≥ 0;
    assume p0 = p; havoc p; assume p = 0;
    assume x0 = x; havoc x; assume x = 0;
    assert p = x * m ∧ x ≤ n;
    havoc x; havoc p; assume p = x * m ∧ x ≤ n,
      WP(assume x < n;
          assume x1 = x; havoc x; assume x = x1 + 1;
          assume p1 = p; havoc p; assume p = p1 + m;
          assert p = x * m ∧ x ≤ n; assume false, p = n * m)
    ∧ x ≥ n ⇒ p = n * m)
```

Now for the loop body, where we have another sequential composition with its last assume false:

```
WP( assume n ≥ 0;
    assume p0 = p; havoc p; assume p = 0;
    assume x0 = x; havoc x; assume x = 0;
    assert p = x * m ∧ x ≤ n;
    havoc x; havoc p; assume p = x * m ∧ x ≤ n,
      WP(assume x < n;
          assume x1 = x; havoc x; assume x = x1 + 1;
          assume p1 = p; havoc p; assume p = p1 + m;
          assert p = x * m ∧ x ≤ n; WP(assume false, p = n * m))
    ∧ x ≥ n ⇒ p = n * m)
```

Let's process the assume false and understand how that stops considering what happens going off the end of the loop body.

```
WP( assume n ≥ 0;
    assume p0 = p; havoc p; assume p = 0;
    assume x0 = x; havoc x; assume x = 0;
    assert p = x * m ∧ x ≤ n;
    havoc x; havoc p; assume p = x * m ∧ x ≤ n,
      WP(assume x < n;
          assume x1 = x; havoc x; assume x = x1 + 1;
          assume p1 = p; havoc p; assume p = p1 + m;
          assert p = x * m ∧ x ≤ n, false ⇒ p = n * m)
    ∧ x ≥ n ⇒ p = n * m)
```

So now B for the WP for the loop body is false $\Rightarrow \ldots$: we don't care what happens after the end of the loop body, we just need to show all the assertions before the end of the loop. That is,

```
WP( assume n ≥ 0;
    assume p0 = p; havoc p; assume p = 0;
    assume x0 = x; havoc x; assume x = 0;
    assert p = x * m ∧ x ≤ n;
    havoc x; havoc p; assume p = x * m ∧ x ≤ n,
      WP(assume x < n;
          assume x1 = x; havoc x; assume x = x1 + 1;
          assume p1 = p; havoc p; assume p = p1 + m;
          assert p = x * m ∧ x ≤ n, true)
    ∧ x ≥ n ⇒ p = n * m)
```

Continuing inside the loop body and applying sequential composition and assert,

```
WP( assume n ≥ 0;
    assume p0 = p; havoc p; assume p = 0;
    assume x0 = x; havoc x; assume x = 0;
```

```
    assert p = x * m ∧ x ≤ n;
    havoc x; havoc p; assume p = x * m ∧ x ≤ n,
      WP(assume x < n;
          assume x1 = x; havoc x; assume x = x1 + 1;
          assume p1 = p; havoc p; assume p = p1 + m,
          p = x * m ∧ x ≤ n)
    ∧ x ≥ n ⇒ p = n * m)
```

We apply another assume:

```
WP( assume n ≥ 0;
    assume p0 = p; havoc p; assume p = 0;
    assume x0 = x; havoc x; assume x = 0;
    assert p = x * m ∧ x ≤ n;
    havoc x; havoc p; assume p = x * m ∧ x ≤ n,
      WP(assume x < n;
          assume x1 = x; havoc x; assume x = x1 + 1;
          assume p1 = p; havoc p,
          p = p1 + m ⇒ p = x * m ∧ x ≤ n)
    ∧ x ≥ n ⇒ p = n * m)
```

Now for the havoc and assume, we introduce fresh variable pa1 and substitute it for p:

```
WP( assume n ≥ 0;
    assume p0 = p; havoc p; assume p = 0;
    assume x0 = x; havoc x; assume x = 0;
    assert p = x * m ∧ x ≤ n;
    havoc x; havoc p; assume p = x * m ∧ x ≤ n,
      WP(assume x < n;
          assume x1 = x; havoc x; assume x = x1 + 1,
          p1 = p ∧ pa1 = p1 + m ⇒ pa1 = x * m ∧ x ≤ n)
    ∧ x ≥ n ⇒ p = n * m)
```

Continuing with x = x1 + 1:

```
WP( assume n ≥ 0;
    assume p0 = p; havoc p; assume p = 0;
    assume x0 = x; havoc x; assume x = 0;
    assert p = x * m ∧ x ≤ n;
    havoc x; havoc p; assume p = x * m ∧ x ≤ n,
      WP(assume x < n;
          assume x1 = x; havoc x,
          x = x1 + 1 ∧ p1 = p ∧ pa1 = p1 + m ⇒ pa1 = x * m ∧ x ≤ n)
    ∧ x ≥ n ⇒ p = n * m)
```

and another havoc, introducing fresh xa1:

```
WP( assume n ≥ 0;
    assume p0 = p; havoc p; assume p = 0;
    assume x0 = x; havoc x; assume x = 0;
    assert p = x * m ∧ x ≤ n;
    havoc x; havoc p; assume p = x * m ∧ x ≤ n,
      WP(assume x < n; assume x1 = x,
          xa1 = x1 + 1 ∧ p1 = p ∧ pa1 = p1 + m ⇒ pa1 = xa1 * m ∧ xa1 ≤ n)
    ∧ x ≥ n ⇒ p = n * m)
```

which leaves us with another `assume x1 = x` to handle; we use ∧ because we are tacking it onto the antecedent of the implication here (essentially, we are simultaneously assuming multiple things).

```
WP( assume n ≥ 0;
    assume p0 = p; havoc p; assume p = 0;
    assume x0 = x; havoc x; assume x = 0;
    assert p = x * m ∧ x ≤ n;
    havoc x; havoc p; assume p = x * m ∧ x ≤ n,
      WP(assume x < n,
          x1 = x ∧ xa1 = x1 + 1 ∧ p1 = p ∧ pa1 = p1 + m ⇒ pa1 = xa1 * m ∧ xa1 ≤ n)
    ∧ x ≥ n ⇒ p = n * m)
```

and finally completing the loop body:

```
WP( assume n ≥ 0;
    assume p0 = p; havoc p; assume p = 0;
    assume x0 = x; havoc x; assume x = 0;
    assert p = x * m ∧ x ≤ n;
    havoc x; havoc p; assume p = x * m ∧ x ≤ n,
      x < n ∧ x1 = x ∧ xa1 = x1 + 1 ∧ p1 = p ∧ pa1 = p1 + m
           ⇒ pa1 = xa1 * m ∧ xa1 ≤ n
    ∧ x ≥ n ⇒ p = n * m)
```

If we continue through the beginning of the method, we get this formula:

```
n ≥ 0 ∧ p0 = p ∧ pa3 = 0 ∧ x0 = x ∧ xa3 = 0 ⇒
  pa3 = xa3 * m ∧ xa3 ≤ n ∧
  (pa2 = xa2 * m ∧ xa2 ≤ n ⇒
    ((xa2 < n ∧ x1 = xa2 ∧ xa1 = x1 + 1 ∧ p1 = pa2 ∧ pa1 = p1 + m) ⇒
        pa1 = xa1 * m ∧ xa1 ≤ n
    ) ∧
    (xa2 ≥ n ⇒ pa2 = n * m)
  )
```

which is equivalent to the conjunction of the three formulas, representing the three things to prove for a loop:

```
n ≥ 0 ∧ p0 = p ∧ pa3 = 0 ∧ x0 = x ∧ xa3 = 0 ⇒
  pa3 = xa3 * m ∧ xa3 ≤ n

n ≥ 0 ∧ p0 = p ∧ pa3 = 0 ∧ x0 = x ∧ xa3 = 0 ∧ pa2 = xa2 * m ∧ xa2 ≤ n ⇒
 (xa2 ≥ n ⇒ pa2 = n * m)

n ≥ 0 ∧ p0 = p ∧ pa3 = 0 ∧ x0 = x ∧ xa3 = 0 ∧ pa2 = xa2 * m ∧ xa2 ≤ n ∧
 x1 = xa2 ∧ xa1 = x1 + 1 ∧ p1 = pa2 ∧ pa1 = p1 + m ⇒ pa1 = xa1 * m ∧ xa1 ≤ n
```

with the first condition being that the loop invariant holds upon entry; the second that it implies the postcondition; and the third that it is preserved by the loop body.

It is possible to simplify these constraints to (or just feed them as-is to a solver):

```
n ≥ 0 ⇒ 0 = 0 * m ∧ 0 leq n

xa2 ≤ n ∧ xa2 ≥ n ⇒ xa2 * m = n * m

xa2 < n ⇒ xa2 * m + m = (xa2 + 1) * m ∧ xa2 + 1 ≤ n
```

which are all valid, proving that the original Hoare triple was also valid.